Autores: Tomás Ferreira (52737) / Tiago Pedroso (52915) / Tiago Inês (54193)

O módulo *Port Expander LCD*(PELCD) implementa a interface com o LCD, fazendo a receção em série da informação enviada pelo módulo de controlo e entregando-a posteriormente ao LCD, conforme representado na Figura 1.

O módulo *PELCD* recebe em série uma mensagem constituída por dez (10) bits de informação. A comunicação com este módulo realiza-se segundo o protocolo ilustrado na Figura 2b, em que o bit RS é o primeiro bit de informação e indica se a mensagem é de controlo ou dados. Os seguintes oito (8) bits contêm os dados a entregar ao LCD. O último bit contém o sinal de E responsável por registar a informação no LCD.

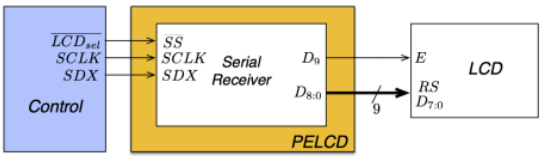


Figura 1: Diagrama de blocos do módulo *Port Expander LCD (PELCD)*

1 Serial Receiver

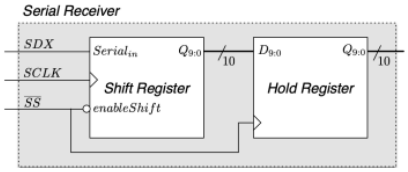
O Serial Receiver é um bloco de hardware responsável por receber dados em formato série (bit a bit) e convertê-los para um formato paralelo (vários bits ao mesmo tempo), que possa ser utilizado por outros módulos do sistema.

No contexto da Ticket Machine, este bloco faz parte de módulos como o Port Expander LCD (PELCD) e o Port Expander Ticket Dispenser (PETD), sendo essencial para interpretar os dados enviados pelo módulo de controlo.

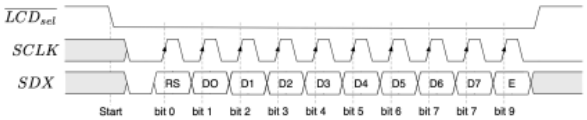
O Serial Receiver é constituído por dois blocos principais, o Shift Register e o Hold Register. O Shift Register recebe os bits um a um, sincronizados com um sinal de clock (SCLK). A cada pulso de clock, o bit recebido é inserido e os anteriores são deslocados e acumula os bits até formar a mensagem completa. Enquanto o Hold Register, mantém os dados estáveis, transferidos do shift register, para serem usados por outros módulos.

O Serial Receiver tem como processo de execução, o seguinte:

1. O emissor inicia a comunicação (condição de start);
2. Os bits chegam um de cada vez pela linha de dados;
3. A cada subida do clock (SCLK), o Serial Receiver lê um bit;
4. O shift register armazena os bits sequencialmente;
5. Quando a mensagem está completa, os dados passam para o hold register;
6. Os dados ficam disponíveis para processamento;



(a) Diagrama de blocos Serial Receiver



(b) Protocolo de comunicação com o módulo Port Expander LCD (PELCD)

Figura 2: Bloco *Key Decode*

A descrição hardware do bloco *Serial Receiver* em *VHDL* encontra-se no Anexo A.3.

2 Conclusões

O Serial Receiver tem como função, receber uma sequência de bits através de uma linha de comunicação série, reconstruir essa sequência num conjunto de dados completo e disponibilizar esses dados para outros blocos (como o LCD ou o dispensador de bilhetes). Ou seja, ele atua como um tradutor entre comunicação série e lógica interna do sistema.

A VHDL

A.1 Hold Register

```
library ieee;
use ieee.std_logic_1164.all;

entity HoldRegister is
    port (
        CLK   : in  std_logic;
        RESET : in  STD_LOGIC;
        D     : in  STD_LOGIC_VECTOR(9 downto 0);
        EN    : in  STD_LOGIC;
        Q     : out std_logic_VECTOR(9 downto 0)
    );
end entity HoldRegister;

architecture logicFunction of HoldRegister is

    component FFD is
        port (
            CLK   : in  std_logic;
            RESET : in  STD_LOGIC;
            SET   : in  std_logic;
            D     : in  STD_LOGIC;
            EN    : in  STD_LOGIC;
            Q     : out std_logic
        );
    end component FFD;

begin

    U0: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(0),
        EN    => EN,
        Q     => Q(0)
    );
    U1: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(1),
        EN    => EN,
        Q     => Q(1)
    );
    U2: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(2),
        EN    => EN,
        Q     => Q(2)
    );
    U3: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
```

```
        D     => D(3),
        EN    => EN,
        Q     => Q(3)
    );
    U4: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(4),
        EN    => EN,
        Q     => Q(4)
    );
    U5: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(5),
        EN    => EN,
        Q     => Q(5)
    );
    U6: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(6),
        EN    => EN,
        Q     => Q(6)
    );
    U7: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(7),
        EN    => EN,
        Q     => Q(7)
    );
    U8: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(8),
        EN    => EN,
        Q     => Q(8)
    );
    U9: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(9),
        EN    => EN,
        Q     => Q(9)
    );

end architecture logicFunction;
```

A.2 Shift Register

```
library ieee;
use ieee.std_logic_1164.all;

entity ShiftRegister is
    port (
        CLK   : in  std_logic;
        RESET : in  STD_LOGIC;
        S     : in  STD_LOGIC;
        EN    : in  STD_LOGIC;
        Q     : out std_logic_VECTOR(9 downto 0)
    );
end entity ShiftRegister;

architecture logicFunction of ShiftRegister is

    component FFD is
        port (
            CLK   : in  std_logic;
            RESET : in  STD_LOGIC;
            SET   : in  std_logic;
            D     : in  STD_LOGIC;
            EN    : in  STD_LOGIC;
            Q     : out std_logic
        );
    end component FFD;

    signal D : std_logic_vector(9 downto 0);

begin

    U0: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => S,
        EN    => EN,
        Q     => D(0)
    );
    U1: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(0),
        EN    => EN,
        Q     => D(1)
    );
    U2: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(1),
        EN    => EN,
        Q     => D(2)
    );
    U3: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
```

```
        D     => D(2),
        EN    => EN,
        Q     => D(3)
    );
    U4: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(3),
        EN    => EN,
        Q     => D(4)
    );
    U5: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(4),
        EN    => EN,
        Q     => D(5)
    );
    U6: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(5),
        EN    => EN,
        Q     => D(6)
    );
    U7: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(6),
        EN    => EN,
        Q     => D(7)
    );
    U8: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(7),
        EN    => EN,
        Q     => D(8)
    );
    U9: component FFD
    port map (
        CLK   => CLK,
        RESET => RESET,
        set   => '0',
        D     => D(8),
        EN    => EN,
        Q     => D(9)
    );

    Q <= D;

end architecture logicFunction;
```

A.3 Serial Receiver

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialReceiver is
    port (
        SDX, CLK, SS, RESET : in  std_logic;
        Q                           : out std_logic_vector(9 downto 0)
    );
end entity SerialReceiver;

architecture logicFunction of SerialReceiver is

    component ShiftRegister is
        port (
            CLK   : in  std_logic;
            RESET : in  STD_LOGIC;
            S     : in  STD_LOGIC;
            EN    : in  STD_LOGIC;
            Q     : out std_logic_VECTOR(9 downto 0)
        );
    end component ShiftRegister;

    component HoldRegister is
        port (
            CLK   : in  std_logic;
            RESET : in  STD_LOGIC;
            D     : in  STD_LOGIC_VECTOR(9 downto 0);
            EN    : in  STD_LOGIC;
            Q     : out std_logic_VECTOR(9 downto 0)
        );
    end component HoldRegister;

    signal shift_out : std_logic_vector(9 downto 0);
         signal not_SS                : std_logic;
begin

         not_SS <= not SS;

    shift: component ShiftRegister
    port map (
        CLK   => CLK,
        RESET => RESET,
        S     => SDX,
        EN    => not_SS,
        Q     => shift_out
    );

    hold: component HoldRegister
    port map (
        CLK   => CLK,
        RESET => RESET,
        D     => shift_out,
        EN    => SS,
        Q     => Q
    );

end architecture logicFunction;
```

B *Kotlin*

B.1 LCD

Algoritmo 1: LCD - Hardware Abstract Layer

```
object LCD {
    private const val LINES = 2
    private const val COLS = 16

    private const val RS_MASK = 0x1
    private const val E_MASK = 0x200

    private fun writeByteSerial(rs: Boolean, data: Int) {
        var frame = (data and 0xFF) shl 1
        if (rs) frame = frame or RS_MASK

        SerialEmitter.send(SerialEmitter.Peripheral.LCD, frame, 10)
        SerialEmitter.send(SerialEmitter.Peripheral.LCD, frame or E_MASK, 10)
        SerialEmitter.send(SerialEmitter.Peripheral.LCD, frame, 10)
    }

    private fun writeByte(rs: Boolean, data: Int) {
        writeByteSerial(rs, data)
    }

    private fun writeCMD(data: Int) {
        writeByte(false, data)

        if (data == 0b00000001) Thread.sleep(2L)
        else Thread.sleep(1L)
    }

    private fun writeDATA(data: Int) {
        writeByte(true, data)
        Thread.sleep(1L)
    }

    fun init() {
        SerialEmitter.init()
        Thread.sleep(20)

        initSequence()

        initCommands()
    }

    fun initSequence() {
        repeat(3) {
            writeByte(false, 0x30) // 0x30 para que ocorra um erro de 8 bits
            Thread.sleep(1)
        }
    }

    fun initCommands() {
        writeCMD(0x38) // 8 bits
        writeCMD(0x08) // display off
        writeCMD(0x01) // clear
        Thread.sleep(5)
        writeCMD(0x06) // entry mode set
        writeCMD(0x0C) // display on
    }

    fun writeChar(c: Char) {
        if (c != '\u0000') writeDATA(c.code)
    }

    fun write(text: String) {
        for (char in text) {
            writeChar(char)
        }
    }
```

```
    fun cursor(line: Int, column: Int) {
        val l = line.coerceIn(0, LINES - 1)
        val c = column.coerceIn(0, COLS - 1)
        val address = if (l == 0) c else 0x40 + c
        writeCMD(0x80 or address)
    }

    fun clear() {
        writeCMD(0x01)
    }
}
```

B.2 Serial Emitter

Algoritmo 2: Serial Emitter

```
object SerialEmitter {
    enum class Peripheral {LCD, TICKET}

    private const val SDX_MASK = 0x01
    private const val SCLK_MASK = 0x02
    private const val LCD_MASK = 0x04
    private const val SSC_MASK = 0x03

    fun init() {
        HAL.init()
        HAL.clrBits(SDX_MASK or SCLK_MASK or LCD_MASK or SSC_MASK)
    }

    fun send(addr: Peripheral, data: Int, size: Int) {
        var _data = data

        HAL.clrBits(SCLK_MASK)

        if (addr == Peripheral.LCD) {
            HAL.clrBits(LCD_MASK)
        } else if (addr == Peripheral.TICKET) {
            HAL.clrBits(SSC_MASK)
        }

        for (i in 0 until size) {
            HAL.writeBits(SDX_MASK, _data)

            _data = _data.shr(1)
            HAL.setBits(SCLK_MASK)
            HAL.clrBits(SCLK_MASK)
        }

        if (addr == Peripheral.LCD) {
            HAL.setBits(LCD_MASK)
            HAL.clrBits(LCD_MASK)
        } else if (addr == Peripheral.TICKET) {
            HAL.setBits(SSC_MASK)
            HAL.clrBits(SSC_MASK)
        }
    }
}
```

B.3 TUI

Algoritmo 3: TUI

```
object TUI {
    fun init() {
        HAL.init()
        KBD.init()
        LCD.init()
        LCD.clear()
    }
```

```
    fun clear() {
        LCD.clear()
    }

    fun cursor(line: Int, col: Int) {
        LCD.cursor(line, col)
    }

    fun writeChar(char: Char) {
        LCD.writeChar(char)
    }

    fun readKey(timeout: Long = 5000L): Char {
        return KBD.waitKey(timeout)
    }

    fun echo() {
        var line = 0
        var col = 0

        while (true) {
            val key = readKey()

            if (key == KBD.Key.KEY_NONE.char) continue

            when (key) {
                '*' -> {
                    clear()
                    line = 0
                    col = 0
                }

                '#' -> {
                    line = (line + 1) % 2
                    col = 0
                    cursor(line, col)
                }

                else -> {
                    cursor(line, col)
                    writeChar(key)
                    col++

                    if (col >= 16) {
                        col = 0
                        line = (line + 1) % 2
                    }
                }
            }
        }
    }
}
```